ITU-MiniTwit - Group e - Report

Frederik Hørup , Marie Johansen , Nikolej Lundquist , Sara Bagger , Vitus Brodersen <>

May 12, 2026

Contents

1 Introduction **1**

2 System **1**
2.1 Architecture and Design 1
2.2 Dependencies 1
2.3 System States 1

3 Process **2**
3.1 CI/CD 2
3.2 Monitoring 2
3.3 Logging 3
3.4 Security 3
3.5 Availability and Scaling 4

4 Reflection **4**

5 Use of Generative AI **4**

1 Introduction

Authors:

2 System

2.1 Architecture and Design

Authors:

2.2 Dependencies

Authors:

2.3 System States

Authors:

3 Process

3.1 CI/CD

Authors: Vitus

Continuous integration and deployment for this project happens in a semi-automatic set of parallel steps. When a developer creates a pull request on the github repository, multiple actions and third-party tools begin checking the validity of the request. Below are the steps:

- Static analysis and security github action Is a compound action for linting, formatting, and security:
  - Hadolint: Dockerfile Linter

  Checks the linked dockerfile for linting issues
  - CSHarpier: C# Formatter

  Checks the entire C# codebase for formatting errors, including spaces
  - Roslyn: C# Linter

  Checks entire codebase for linting, especially typesafety
  - CodeQL: Static Code Security

  Checks code against a database of known security flaws
  - Docker Scout: Docker Image Vulnerability scanner

  Checks the docker image on the DockerHub for known vulnerabilities
- Codacy static analysis

  Third-party Static analysis of the code. includes recent real-world breaches
- SonarCloud code analysis

  Another code analysis program, with focus on general code quality
- Build and Test action

  Builds the program and executes the test suite on pushes to a PR.
- Staging and Deployment action

  Stages the system and adds it to the deployment
- Review and Automatic release A Pull Request can only be merged to main upon the review of another developer, per git requirements.

  Releases are automatically done weekly at Tuesdays 08:00 UTC (10:00 danish summer time)

3.2 Monitoring

Authors: Marie The systems monitorting is setup using the open-source monitoring system Prometheus in colaboration with Grafana for visualizing and quering the metrics. The `app.MapMetrics();` and `app.UseHttpMetrics();` middleware were added to the pipline. `app.MapMetrics();` exposes the HTTP endpoint for Prometheus to scrape and `app.UseHttpMetrics();` collect Prometheus metrics for processed HTTP requests (from documentation of UseHttpMetrics).

The monitorting is pull based as the application exposes metrics which are then pulled by Prometheus.

The monitoring as been split up into two dashboards; application metrics and infrastructure metrics. Application metrics focueses mostly on request rates and displays: CPU Usage in Seconds and HTTP Request Recieved in total as well as split into different types of requests

Infratructure metrics focuses on the server side and displays dashboards contaning information about: memory usage, CPU usage and process uptime

Reactive monitoring because we provide a small amount of dashboard that are mostly operationally-focused and the broad focus has been on measuring availability. The monitoring has however not moved towards monitoring data to measure user experience or that the business side would benefit from.

There are many ways monitoring could have been improved. For one, database monitoring would have been especially beneficial both both for the operational side and to provide metrics for the business side e.g. number of users in the system

Lastly, the monitoring dashboards provided by Digital Ocean to monitor the VMs has been regullary used.

3.3 Logging

Authors:

3.4 Security

Authors: Sara

We applied several security hardening measures to our system across infrastructure, network, CI/CD, and container configuration.

Reverse proxy/TLS. First, we deployed a Nginx reverse proxy to the application and enabled HTTPS using TLS certificates. The reverse proxy terminates incoming HTTPS traffic and forwards requests internally to the application containers. This improves security by encrypting communication between clients and the server and by reducing direct exposure of the application itself. We used Let's Encrypt certificates together with automatic renewal mechanisms to avoid manual certificate management.

Firewall. To protect the servers themselves, we configured a UFW software firewall. We followed the principle of least privilege by allowing only required traffic such as SSH and HTTP/HTTPS while denying unnecessary incoming connections. This was important to avoid unintentionally exposing services externally if firewall rules are misconfigured. We also considered firewall logging to detect suspicious or blocked traffic patterns, but because of lack of time, this was omitted.

CI/CD. In the CI/CD pipeline, we integrated automated security analysis tools to support a shift-left security approach, where vulnerabilities are detected before deployment. We added GitHub CodeQL analysis to statically scan the application source code for known security vulnerabilities and insecure coding patterns. Additionally, we integrated Docker Scout to scan container images for known common vulnerabilities and exposures and vulnerable dependencies. The pipeline was configured to fail on critical vulnerabilities, preventing insecure images from being deployed automatically.

Docker hardened images. We also hardened our Docker environment by switching several production services to Docker Hardened Images (DHI). Specifically, we replaced standard Grafana, Prometheus, and ASP.NET images with hardened variants from dhi.io. These images are designed to reduce attack surfaces by minimizing unnecessary packages and dependencies.

Container execution. Beyond changing base images, we further hardened container execution. For example, Grafana was configured to run as a non-root user instead of running with root privileges.
We also introduced initialization steps to ensure correct permissions on mounted volumes without granting unnecessary privileges to the running application containers.

A key lesson from the course was that security should not rely on a single mechanism. Therefore, our approach combined multiple layers of protection: encrypted communication through TLS, restricted network access through firewalls, automated vulnerability scanning in CI/CD, hardened container images, and safer runtime configurations. This follows a defense-in-depth strategy, where multiple independent security mechanisms reduce the likelihood that a single vulnerability compromises the entire system.

3.5 Availability and Scaling

Authors:

4 Reflection

Authors:

5 Use of Generative AI

Authors: Nikolej

During the development of this project we used ChatGPT in two main ways:

Debugging. Often when encountering unexpected bugs and error messages, we would consult ChatGPT about the cause and potential fixes While it could rarely fix the issues entirely by itself, more often than not, it pointed us in the right direction.

Research. This course presents challenges involving numerous technologies, most of which we were unfamiliar with. As such, ChatGPT was used to summarize lengthy documentation and provide concrete guides tailored to our situation.

Generating boilerplate / configurations. ChatGPT was also used for simple tasks like generating boilerplate code or writing Github Actions Workflow files. For example, the `build-report.yml` workflow, that converts the markdown source into a pdf.

The use of AI has made these tasks easier, spared us many frustrations and saved consiberable time during development. On the flip side, AI may have deprived us the extensive knowledge and experience you gain by, for example, painstaking reading of documentation or spending hours resolving a tiny bug.